ACME++: A Secure Authorization Mechanism for ACME Clients in the Web PKI Ecosystem

Tianyu Zhang
Tsinghua University
Beijing, China
Zhongguancun Labortary
Beijing, China
zhangty23@mails.tsinghua.edu.cn

Han Zhang*
Tsinghua University
Beijing, China
Zhongguancun Labortary
Beijing, China

Yunze Wei
Tsinghua University
Beijing, China

Yahui Li
Tsinghua University
Beijing, China

Xingang Shi
Tsinghua University
Beijing, China
Zhongguancun Labortary
Beijing, China

Jilong Wang
Tsinghua University
Beijing, China
Zhongguancun Labortary
Beijing, China

Xia Yin
Tsinghua University
Beijing, China
Zhongguancun Labortary
Beijing, China

Abstract

The Automatic Certificate Management Environment (ACME) protocol automates the issuance and renewal of secure socket layer certificates, simplifying the management of large-scale certificate deployments. To reduce the load on Certificate Authority (CA) servers, ACME employs a caching mechanism that stores domain validation (DV) results for 30 days. However, this mechanism allows attackers to reuse previously authorized results, potentially bypassing the DV process. In this paper, we introduce the ACME *Authz* Cache Attack, whereby an attacker can obtain fraudulent certificates without domain control. We demonstrate that even the prominent CA, Let's Encrypt, is vulnerable to this attack. To mitigate this, we propose ACME++, an enhanced protocol that binds the client's IP address and a unique identifier to the ACME account, ensuring secure authorization for each new client and effectively preventing the ACME *Authz* Cache Attack. Our implementation of ACME++ shows that it introduces little overhead to the CA server.

CCS Concepts

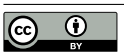
• **Security and privacy** → **Security protocols**; • **Networks** → *Network protocol design.*

Keywords

Web PKI; HTTPS; SSL/TLS; ACME; Fraudulent Certificates; Certificate Transparency; Authorization; Let's Encrypt

ACM Reference Format:
Tianyu Zhang, Han Zhang, Yunze Wei, Yahui Li, Xingang Shi, Jilong Wang, and Xia Yin. 2025. ACME++: A Secure Authorization Mechanism for ACME Clients in the Web PKI Ecosystem. In *Proceedings of the ACM Web Conference 2025 (WWW '25), April 28-May 2, 2025, Sydney, NSW, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3696410.3714763

*Han Zhang is the corresponding author.



WWW '25, April 28-May 2, 2025, Sydney, NSW, Australia

ACM ISBN 979-8-4007-1274-6/25/04
https://doi.org/10.1145/3696410.3714763

1 Introduction

Today, about 96% of Google's web traffic is secured with HTTPS [27, 53], ensuring data integrity and privacy for services like e-commerce and video streaming. These connections are secured by secure socket layer (SSL) certificates managed under the Web Public Key Infrastructure (PKI). Through the domain validation (DV) process, Certification Authorities (CAs) bind domain identities to public keys in SSL certificates. When establishing an HTTPS connection, the web server presents the certificate to the client for identity verification.

For a single web server, requesting and deploying an SSL certificate is straightforward. However, in large enterprises, managing SSL certificates at scale becomes complex. Manual processes for requesting, deploying, and renewing certificates are time-consuming and error-prone. Even a single expired or misconfigured certificate can cause service outages and financial losses [4, 46, 50–52].

To address these challenges, the Automatic Certificate Management Environment (ACME) protocol [5] was introduced and widely adopted by major CAs, automating certificate lifecycle tasks like issuance, renewal, and revocation. A notable example is Let's Encrypt [33], a CA that issues certificates exclusively through ACME. By 2021, Let's Encrypt accounted for nearly 30% of certificates in IPv4 scans and over 80% in Certificate Transparency (CT) logs [22]. Other commercial CAs have also integrated ACME [18], making it an industry standard.

Despite its widespread adoption, the ACME protocol introduces several risks. A primary concern is its caching of domain authorization results after a client requests an SSL certificate. This caching mechanism, originally designed to reduce the workload on CAs, can be exploited by attackers to bypass DV and obtain fraudulent certificates. Such certificates can then be used to conduct man-in-the-middle (MITM) attacks, which present significant threats to both the Web PKI ecosystem and user data privacy. For instance, in 2011, hackers compromised DigiNotar's CA server and issued fraudulent certificates to intercept user traffic intended for Google Gmail services [12, 23, 35]. Similarly, in 2015, Google discovered that MCS Holdings, an Egyptian company, had deployed fraudulent certificates within its proxy devices to intercept traffic [7].

In this paper, we introduce the ACME *Authz* Cache Attack, a comprehensive method that exploits ACME's caching risk to obtain fraudulent certificates without domain control. We demonstrate that an attacker, upon accessing a victim's ACME account credentials, can bypass DV by reusing cached authorization (*Authz*) records. Additionally, we show that attackers can expand their attack surface by identifying *associated domains*, which are domains retrieved from the SSL certificate Subject Alternative Name (SAN) extension discovered in CT logs. We show that Let's Encrypt is vulnerable to this attack, highlighting its widespread impact.

To counter this attack, we propose ACME++, an enhanced version of the ACME protocol designed for easy deployment. ACME++ strengthens security by binding client IP addresses and unique identifiers to ACME accounts. When a certificate request is made by a new client, ACME++ requires DV for partial domain names with valid *Authz* records. ACME++ is designed for seamless integration with the existing ACME framework, reusing current objects and message exchange standards to minimize protocol modifications. We implemented ACME++ on both the ACME client and CA server, demonstrating its operational efficiency. Additionally, we show that ACME++ is resilient against various attacks.

The main contributions of this paper are as follows:

- We identify and analyze a risk in ACME's caching mechanism, demonstrating how the reuse of cached *Authz* records can be exploited by attackers.
- We introduce the ACME *Authz* Cache Attack, a novel attack in which an attacker can obtain fraudulent certificates without domain control after acquiring ACME account credentials. We also show that attackers can leverage CT logs to identify *associated domains*, with about half of these domains being valuable for targeting.
- We propose ACME++, an enhanced version of ACME, designed to mitigate the aforementioned attack. ACME++ incorporates additional security checks for ACME clients when interacting with CAs, preventing the exploitation of exposed credentials. Our evaluation of ACME++ demonstrates that, in worst-case scenarios, it results in a minimum 13% increase in processing time and a 5% increase in traffic overhead for CAs, while also proving its resilience against several potential attack vectors.

The remainder of this paper is organized as follows: Section 2 provides background on Web PKI, ACME protocol, and its caching mechanism. Section 3 describes the ACME *Authz* Cache Attack, while Section 4 outlines ACME++. We discuss related works in Section 5, and Section 6 concludes the paper.

2 Background

In this section, we introduce the foundational concepts related to attack vectors and mitigation strategies. We begin with an overview of the Web PKI ecosystem (2.1), focusing on the architecture and roles of each component. Then we delve into the ACME protocol and its caching mechanism (2.2).

2.1 Web PKI

Web PKI is a public key infrastructure framework designed to secure communication between websites and users, preventing data tampering and MITM attacks. As noted in [16], the primary entities in Web PKI include CAs and End Entities (EEs). For a more detailed survey of Web PKI architecture, readers can refer to [13, 14, 24, 30, 40].

Table 1: ACME CA *Directory* and service descriptions.

Service	Service URL	Service Description
newNonce	`/acme/new-nonce`	Obtains a new nonce from the CA to prevent replay attacks.
newAccount	`/acme/new-account`	Registers a new ACME account.
newOrder	`/acme/new-order`	Orders a new certificate from the CA.
newAuthz	`/acme/new-authz`	Creates an *Authz* object for pre-authorization (optional).
revokeCert	`/acme/revoke-cert`	Submits a certificate revocation request to the CA.
keyChange	`/acme/key-change`	Updates the binding of an ACME account key.

The CA plays a central role in the Web PKI framework by managing SSL certificates for all EEs. Web servers, acting as EEs, deploy SSL certificates to establish secure HTTPS connections with clients. In accordance with the X.509 standard [16], SSL certificates bind the *subject* (typically a domain name) to the web server's *public key*, and also include fields such as validity periods, issuer information, and X.509 extensions. Within the Web PKI, EEs' certificates are signed by the CA, and the CA's certificate is signed by a higher-level CA, forming a *chain of trust* that ultimately terminates at a trusted root CA. When a user accesses a website, the browser verifies this certificate chain to ensure the authenticity of the website.

In the Web PKI ecosystem, the primary security concern is the possibility of fraudulent certificates—certificates issued by a CA without proper domain owner confirmation, or those issued through attacks on the DV process [6, 8, 9, 17]. This is particularly problematic as certificate issuance is an end-to-end process, and the public is typically unaware of their existence. To address this issue, Google introduced CT in 2012 [25, 31], a system designed to detect fraudulent SSL certificates by maintaining a publicly auditable, immutable log of all certificates issued by CAs. While CT logs are instrumental in the early detection of fraudulent certificates, they also pose a risk of exposing sensitive information to third parties, such as domain names and business relationships [38, 41, 42].

2.2 ACME Protocol

The ACME protocol [5] automates the entire certificate lifecycle, including request, issuance, renewal, and revocation. For each action, the protocol specifies a unique service URL. CA servers provide clients with the *Directory*, a dictionary-like object where keys represent service types and values are the corresponding URLs. Table 1 lists all available services and their URLs in the CA *Directory*. In addition to the *Directory*, the CA provides resource URLs for clients to retrieve objects. For example, the client can access a *Authz* record via `/acme/authz/<authz_id>`, where `authz_id` is the unique identifier of the *Authz* record.

Figure 1 illustrates the typical process of requesting an SSL certificate using the ACME protocol. The client communicates with the CA server through ACME client software, such as Certbot [19] or acme.sh [1]. Initially, the client submits a *newAccount* request, providing a signed payload that includes an email address and public

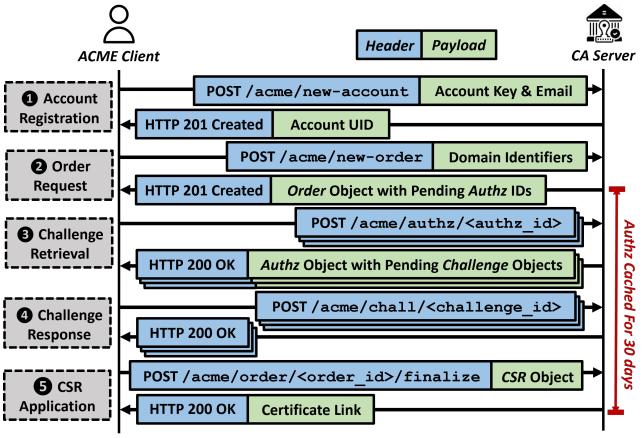


Figure 1: The certificate request process via the ACME protocol. Valid *Authz* records are cached for 30 days after the CA successfully validates the client's domain ownership.

key. Once verified, the CA creates an ACME account and returns a unique account ID (UID). The client then sends a *newOrder* request, specifying the domains for which the certificate is requested.

The CA then initiates the DV process. To validate domain ownership, the CA issues one or more challenges to the client, typically in the form of HTTP-01 or DNS-01 challenges [32]. The client must complete these challenges before the order can proceed. Upon successful completion, the CA caches the DV result, known as an authorization (*Authz*) record, for a period of 30 days. This caching mechanism facilitates the quicker reissuance of certificates for the same domain without requiring re-validation. Finally, the client submits a Certificate Signing Request (CSR) and receives the final SSL certificate link to download and deploy.

We show that this caching mechanism described above introduces a critical security risk. As demonstrated in Section 3, an attacker can exploit the cached *Authz* to obtain fraudulent certificates using the victim's ACME account credentials. Under normal conditions, the CA would be unable to detect such an attack.

3 ACME *Authz* Cache Attack

In this section, we introduce a novel attack exploiting the ACME protocol's caching mechanism to acquire fraudulent certificates, based on the observation that cached *Authz* records can be considered as the ownership credentials of the domains. We first outline the attack assumption (3.1), followed by a detailed description of the attack methodology (3.2). Finally, we present real-world experiments, demonstrating the vulnerability of Let's Encrypt to this attack (3.3), and analyzing the number of *associated domains* for given target websites (3.4).

3.1 Attack Assumption

We assume the following conditions for the attacker and victim:

Attacker Assumptions.

(1) The attacker acquires the victim's ACME account credentials by either exploiting server-side vulnerabilities (such as Log4j [3] or path traversal flaws [37]), or through improper handling of keys by web operators, such as insecure key sharing [10]. In Appendix A, we elaborate on how these scenarios are prevalent across the Internet and how the attacker can exploit them to gain access to the account credentials.

(2) The attacker is aware of at least one domain service deployed on the victim's server as the target domain.

(3) The attacker controls a single host and can operate from anywhere globally, typically far away from the victim.

Victim Assumptions.

(1) The victim's server stores its ACME account credentials locally on the server.

(2) The victim has requested certificates in the last 30 days so that there are valid *Authz* records cached by the CA.

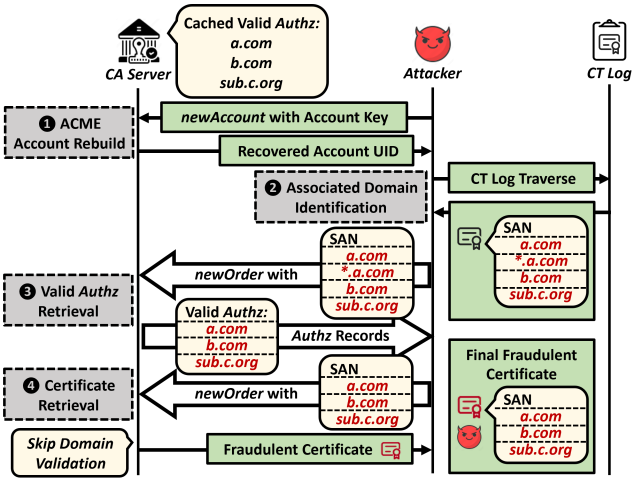


Figure 2: ACME *Authz* Cache Attack in four steps.

3.2 Attack Methodology

Figure 2 illustrates this attack through an example. The attack consists of four main steps: rebuilding the victim's account profile, identifying *associated domains*, retrieving valid *Authz* records, and obtaining fraudulent certificates. In this example, the victim has an ACME account that has requested certificates for `a.com`, `b.com`, and `sub.c.org`. These domain *Authz* records remain valid throughout the attack. The attacker knows `a.com` as the target domain.

3.2.1 ACME Account Profile Rebuild. The first step of the attack is to reconstruct the victim's ACME account profile using the obtained account credentials. If the attacker has full access to the credentials, the profile can be easily restored by setting the correct storage path. However, if only the account key pair is obtained (e.g., via insecure key reuse), the account UID can be recovered by sending a *newAccount* request with the account public key and private key signature to the CA, provided that the CA supports this method. Alternatively, the attacker may resort to brute-forcing the UID if the CA assigns UIDs sequentially, rather than using random strings as recommended by the RFC [5] (see Appendix B for further details).

3.2.2 Associated Domain Identification. The next step is to identify as many *associated domains* as possible. A domain (including wildcard domains) is considered an *associated domain* if it appears in the same SSL certificate as the target website domain. Website

operators often group multiple domains hosted on the same server into a single certificate to reduce operational costs and complexity. Through the ACME protocol, *Authz* records for these *associated domains* and the target domain are cached under the same ACME account, making them high-value targets for the attacker seeking to expand their attack surface.

To discover *associated domains*, the attacker can search through certificates archived in public CT logs. In this attack, the attacker can identify certificates issued for the target domain within the past 30 days and search for *associated domains* in the certificates' SAN extensions. In Figure 2, the *associated domains* of the target domain `a.com` include `*.a.com`, `www.a.com`, `b.com`, and `sub.c.org`.

3.2.3 Valid Authz Record Retrieval. With the account profile and *associated domains* in hand, the attacker can attempt to discover valid *Authz* records. The attacker submits a *newOrder* request containing the target domain along with all discovered *associated domains*. The CA responds with an *Order* object, which includes a list of *Authz* URLs for all the attached domains. The attacker can then traverse these *Authz* URLs to check the status of the *Authz* records. A valid status indicates a valid *Authz* record. In this illustration, the *newOrder* request includes `a.com`, `*.a.com`, `www.a.com`, `b.com`, and `sub.c.org`. Of these, only `a.com`, `b.com`, and `sub.c.org` have valid records, while `*.a.com` and `www.a.com` have a "pending" status.

3.2.4 Fraudulent Certificate Retrieval. In the final step, the attacker obtains a fraudulent certificate by submitting another *newOrder* request, including all domains with valid *Authz* records, thereby bypassing the DV process.[1] In this example, the attacker successfully obtains a fraudulent certificate for `a.com`, `b.com`, and `sub.c.org`.

3.3 Evaluation Against Let's Encrypt

3.3.1 Setup. We evaluate the proposed attack (introduced in 3.1 and 3.2) against Let's Encrypt using our test domain `sparklestar.work`. The victim server, hosting this domain, uses Certbot [19] as the ACME client to request certificates from Let's Encrypt for both the domain and its wildcard subdomain `*.sparklestar.work`. The server is configured with a misconfiguration in the Nginx alias, as described in [28], making it vulnerable to a path traversal attack. Additionally, the test domain service is deployed with the "root" account, granting the web service root access permissions. Another server is set up as the attacker. By exploiting this path traversal vulnerability with a carefully constructed path, such as `"sparklestar.work/i../../etc/letsencrypt"`, the attacker can access files in the directory where the ACME account credentials are stored, thereby obtaining the full credentials.

3.3.2 Issuing Fraudulent Certificates. Following the attack procedure, the attacker first reconstructs the victim's ACME account profile on the attack server. The attacker then locates the victim's requested certificates in Google's CT log (argon2024 [26]) and identifies the *associated domain* `*.sparklestar.work` in the SAN extension. Next, the attacker submits a *newOrder* request to Let's Encrypt for the test domain and its *associated domain*. Due to the cached, valid *Authz* records on the CA server, the attacker successfully obtains a certificate for the victim's domains without undergoing any DV challenges. This confirms the feasibility of the attack.

3.3.3 Adding Phishing Domains. We extend our evaluation to examine whether an attacker could append phishing domains to the obtained fraudulent certificate. In such cases, the phishing domains would be more challenging to distinguish, as they would appear alongside legitimate ones on the same SSL certificate. For this experiment, the attacker registers another test domain, `sparkle-star.org`, and includes it in the *newOrder* request. As anticipated, Let's Encrypt only requires DV for this new domain, bypassing DV for `sparklestar.work` and `*.sparklestar.work`. The attacker successfully obtains a certificate with `sparkle-star.org`, `sparklestar.work`, and `*.sparklestar.work` appearing in the same SAN list.

3.4 *Associated Domain* Analysis

In this section, we assess the attack surface by investigating the number of *associated domains* that can be identified for a given target website. Using domains from Cisco's Top-1M website list [48] as test subjects, we conduct a search for certificates issued in September 2024 containing these domains in two separate CT logs: Cloudflare Nimbus [15] and Sectigo Sabre [43]. We select these two CT logs as they are among the most widely used and could serve as representative samples. For each certificate identified, we extract all *associated domains* listed in the SAN field.

The preliminary results are summarized in Table 2, which shows that 135,569 and 117,993 target domains from the Top-1M websites have *associated domains* found in Nimbus and Sabre, respectively. We plot the CDF of the number of *associated domains* in Figure 3a. It can be observed that over 40% of the target domains have at least one *associated domain* beyond themselves in Sabre, and about 30% in Nimbus. Both CT logs exhibit a long-tail distribution, suggesting that a few websites have an exceptionally large number of *associated domains*. Several target domains (e.g., those owned by CDNs) even have over 1 million *associated domains*, which significantly increases the potential attack surface.

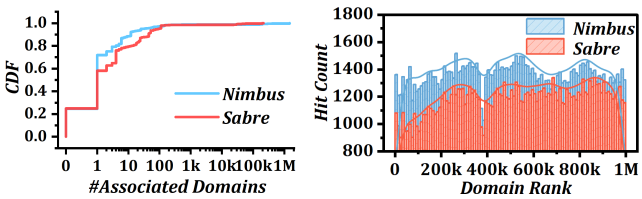
We then analyze the relationship between the presence of *associated domains* and the rank of the target websites. Figure 3b shows the distribution in Nimbus and Sabre, revealing similar trends across the rank spectrum. This suggests that both highly ranked and less prominent domains have extendable attack surfaces.

We further evaluate the number of *associated domains* potentially managed under the same ACME account as the target domain, which could indicate high-value targets for the ACME *Authz* Cache Attack. To do so, we compare the JARM TLS fingerprints [2] of the *associated domains* with that of the target domain. Domains sharing the same JARM fingerprint are likely managed by the same server operator, increasing the likelihood they belong to the same ACME account. Figure 4 presents the JARM hit results. The x-axis shows the group index, where each group corresponds to a specific range of the number of *associated domains*, with group lengths chosen to balance the number of target domains in each group. The CDF line represents the cumulative percentage of target domains in each group, while the boxplots display the JARM hit rate within the 25-75 percentile for each group.

[1] In fact, the attacker can obtain more than one fraudulent certificate. The number of certificates an attacker can request is limited by the CA's rate limits. For Let's Encrypt, the limit is 50 certificates per registered domain per week [20].

Table 2: Overview of *associated domain* discovery results for Cisco Top-1M website domains. All certificates were issued in September 2024.

CT Log Name	Nimbus [15]	Sabre [43]
# Certificates with Top-1M Domains	1,195,862	663,813
# Top-1M Domains Covered	135,569	117,993
# *Associated Domains* Discovered	7,314,093	1,784,209
# Domains with SSL Deployed	842,671	525,596



(a) CDF graph of the number of *associated domains* (x-axis in log scale). **(b) Rank distribution of websites that have *associated domains* (bin size = 10K).**

Figure 3: *Associated domain* distribution results.

Figures 4a and 4b show that, across all but the last groups in Nimbus and Sabre, the median JARM hit rates consistently exceed 50%, indicating that more than half of the *associated domains* identified in each CT log are highly valuable for attack success. Although the last group exhibits greater variability and a lower median, it represents a very small fraction of target domains (<0.01%), which has minimal impact on the overall findings. Furthermore, for nearly all target domains in the first group, where there are only one or two *associated domains*, the median JARM hit rate reaches 100%, suggesting a high likelihood that these *associated domains* are particularly vulnerable to attacks.

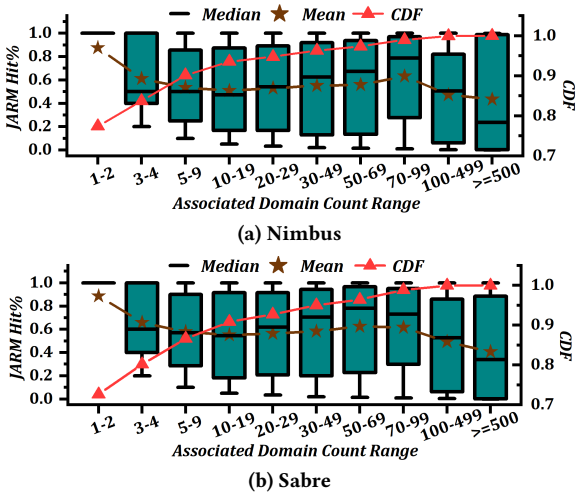
Takeaways. The *associated domain* analysis reveals that an attacker can identify numerous target domains across a wide range of ranks in each CT log, with *associated domain* counts ranging from 1 to over 1 million. On average, about half of these *associated domains* are likely managed under the same ACME account, making them valuable targets in the third step of the attack (see 3.2.3).

3.5 Ethics and Disclosure

Section 3 outlines how malicious attackers may exploit vulnerabilities within the ACME protocol to obtain fraudulent SSL certificates without possessing domain control. All experiments involving Let's Encrypt were conducted within a controlled environment, utilizing test domains that were registered by us and servers intentionally configured with known vulnerabilities to facilitate evaluation. For the analysis of *associated domains*, all certificate data was collected from publicly available CT logs. During the JARM TLS fingerprinting scan, a maximum of 10 Client Hello messages were sent to each target server, ensuring the scanning process did not impose excessive load on the servers.

We disclosed our proposed attack and findings to four major CAs: Let's Encrypt, DigiCert, Google Trust Services, and Sectigo.

Let's Encrypt acknowledged the reported attack but emphasized that it is the ACME client's responsibility to protect the account credentials. As a result, they decided to close the report.



(a) Nimbus

(b) Sabre

Figure 4: JARM hit percentages for *associated domains* across target domain groups, with CDF lines representing target domain counts, in both CT logs.

DigiCert and Google Trust Services acknowledged the issue and forwarded it to their technical support teams but have not provided further follow-up.

Sectigo stated that they do not use the ACME *Authz* cache mechanism for certificate issuance and are therefore not vulnerable to this attack. Consequently, no further action was taken.

Our primary objective is to raise awareness about the potential risks associated with the ACME protocol, and to contribute to the enhancement of overall security within the Web PKI community. We advocate for the adoption of mitigations, such as ACME++, as discussed in the next section.

4 ACME++: Secure ACME Client Authorization

The cache mechanism in the ACME protocol suggests that possessing an ACME account alone is sufficient to prove domain control, which underpins the ACME *Authz* Cache Attack. To address this issue, we propose **ACME++**, an enhanced version of the ACME protocol that introduces additional authorization steps for clients sharing the same ACME account. In this section, we outline the design principles and details of ACME++ (4.1-4.3), followed by an evaluation of its effectiveness (4.4).

4.1 Design Principles

ACME++ aims to strengthen ACME client authorization while maintaining full compatibility with existing Web PKI infrastructure and CA configurations. Its design is guided by the following principles:

- **Robust Security.** ACME++ mitigates the ACME *Authz* Cache Attack without introducing new vulnerabilities, such as susceptibility to brute-force attacks. It ensures resilience against both current and emerging threats, thereby protecting the certificate issuance process.
- **Minimal Overhead for CAs.** ACME++ retains the original ACME cache mechanism to avoid redundant DV processes. Any additional authorization steps are lightweight, ensuring minimal impact on CA performance.

- **Ease of Deployment.** Unlike existing solutions that require manual updates and inspections to address ACME account credentials exposure, ACME++ requires only a single client-side update, simplifying deployment and ongoing maintenance.
- **Consistency with Web PKI.** ACME++ enhances the existing ACME protocol without modifying the Web PKI structure, ensuring full compatibility and enabling the introduction of new features without necessitating architectural changes.

4.2 *Client Authz* Object

ACME++ introduces the *Client Authz* object, a client authorization record (the format is detailed in Table 3) designed to secure the certificate issuance process. Each *Client Authz* object associates a unique `identifier` with the client's IP address and a client-specific ID, binding an ACME client to a particular ACME account. The IP address corresponds to the host running the ACME client software, while the client ID is a randomly generated string provided by the client. The IP address binding scheme helps differentiate legitimate clients from attackers, thus enhancing resilience against various attack vectors (see Section 4.5). The unique ID can be used to distinguish between different clients sharing the same ACME account and IP address (e.g., multiple hosts behind a NAT). This provides more precise control over domain authorization within the same ACME account, even if the account is shared by different users.

Similar to the *Authz* record in ACME, the CA caches the *Client Authz* object with a `status` of "valid" for 30 days, provided that the associated `challenges` are completed. The `challenges` field contains multiple *Challenge* objects that the client must fulfill. The details of challenge generation are discussed in Section 4.3. In addition to the *Client Authz* object, ACME++ introduces a new resource URL, `/acme/client-authz/<authz_id>`, allowing the client to retrieve the *Client Authz* object using the specified `authz_id`.

4.3 Message Exchange Flow

The ACME++ workflow for requesting a new certificate is illustrated in Figure 5. When an ACME client initiates a *newOrder* request with the CA server, it includes its host IP address, client ID, and ACME account credentials. Upon receiving the request, the CA performs two verification steps: (1) checking if the request's source IP address matches the client's host IP address, and (2) verifying the existence of a valid *Client Authz* record for the client. If both conditions are satisfied, the CA proceeds with certificate issuance according to the standard ACME protocol. If the IP addresses in step (1) do not match, the CA terminates the connection, as this discrepancy may indicate that the ACME account credentials have been compromised, potentially triggering the ACME *Authz* Cache Attack. If no valid *Client Authz* record exists, the CA creates a new *Client Authz* object for the client, marks its `status` as "pending," and returns the object ID to the client.

When generating the *Client Authz* object, a list of challenges is created to populate the `challenges` field. The challenge generation follows a structured approach: the CA first retrieves all valid *Authz* records associated with the ACME account and randomly selects $n = \lceil \log(N+1) \rceil$ domain names, where N denotes the total number of domains with valid *Authz* records for the account. The

Table 3: *Client Authz* object fields.

Field Name	Field Description
`status`	The current authorization status of the client (e.g., "valid," "pending").
`expires`	The expiration time of the client authorization.
`identifier`	A unique identifier for the client, consisting of the client IP and a randomly generated client ID.
`challenges`	A list of *Challenge* objects for client authorization.

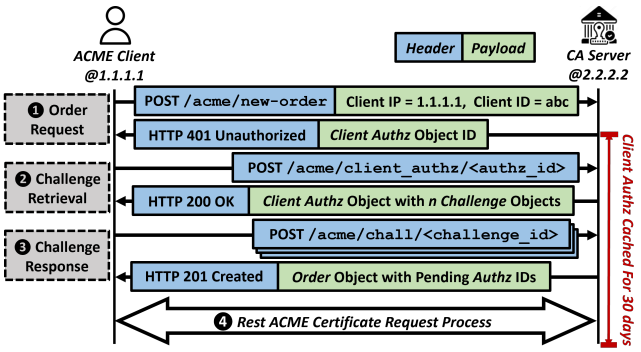


Figure 5: ACME++ workflow and message exchange for requesting a new SSL certificate.

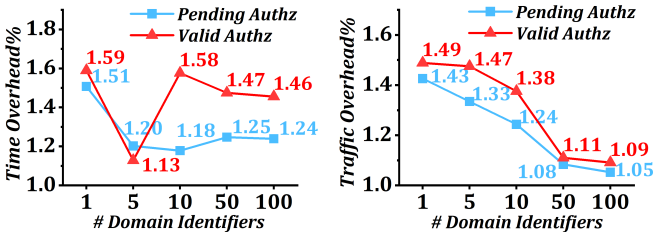
CA generates *n Challenge* objects for these selected domains and fills the `challenges` field.

Next, the client sends a POST-as-GET request to `/acme/client-authz/<authz_id>` to retrieve the *Client Authz* object, which contains pending *Challenge* objects, either HTTP-01 or DNS-01 types. After successfully completing all challenges, the client notifies the CA at `/acme/chall/<challenge_id>` for each challenge that has been completed. On the CA side, after verifying these notifications, the ACME client is authorized, so the CA updates the *Client Authz* object status to "valid" and then proceeds with certificate issuance.

Compatibility Analysis. Throughout the message exchange flow, only the ACME client and the CA are the communicating entities. No third parties are introduced during this process, which ensures compatibility with the Web PKI architecture. Furthermore, ACME++ reuses the ACME *Challenge* objects for client verification, and the final phase of ACME++ mirrors that of ACME. This results in minimal modifications to the existing ACME protocol. As a result, the client side can continue using the same ACME client software, with only updates required for the new components.

4.4 Overhead Evaluation

In this section, we evaluate the overhead of the ACME++ protocol in terms of the CA server. In typical use cases, such as a web server with a fixed public IP address that periodically requests certificates, the occurrence of ACME++ overhead is infrequent due to the 30-day caching of the *Client Authz* record. This caching mechanism ensures that the client only needs to complete a new authorization if the cached record expires. This is particularly advantageous in scenarios involving high-frequency certificate requests, as it minimizes the need for repeated interactions with the CA. Furthermore, the logarithmic selection of domain challenges significantly reduces



(a) Time overhead. (b) Traffic overhead.

Figure 6: ACME++ overhead on the CA server, based on simulation results in worst-case scenarios.

the challenge overhead in large-scale deployments. For example, if a client manages $N = 10,000$ domains, it would only need to complete $n = \lceil \log(10,000 + 1) \rceil = 14$ challenges, which accounts for only 0.14% of the total domains.

However, when clients operate with dynamic IP addresses (e.g., within a campus network), valid *Client Authz* checking may fail with each certificate request, necessitating re-authorization and introducing additional CA overhead for each request. To evaluate this, we simulate two extreme scenarios to assess the certificate issuance overhead.

In both scenarios, the ACME client sends a *newOrder* request to the CA with a set of domain identifiers. In the first scenario (*Pending Authz*), none of the domain identifiers have valid *Authz* records cached in the CA server, requiring both the ACME++ client authorization and the DV processes. In the second scenario (*Valid Authz*), all domain identifiers have valid *Authz* records cached in the CA server, requiring only the client authorization step.

4.4.1 Setup. We implement both the ACME and ACME++ protocols in Python for the CA server and ACME client, following the guidelines outlined in the RFC documentation [5]. The implementation simulates the message exchange process involved in requesting an SSL certificate. We set up two servers: one acting as the client, hosting our test domain service, and the other as the CA. To simulate real-world certificate issuance between the CA and the client, we deploy the client and server on different ISP networks. The CA server is configured to use HTTP-01 challenges during the DV process, while a local DNS resolver is employed to mitigate the effects of DNS resolution time fluctuations caused by the network. To standardize the time for challenge completion on the client side, we assume a fixed duration of 1 second before the client sends notifications back to the CA.

We evaluate the system using two metrics: (1) **Time Overhead**, measured as the ratio of the duration from the client's *newOrder* request to the final certificate issuance; and (2) **Traffic Overhead**, defined as the ratio of total bytes transmitted between the client and the CA server for a single certificate request in ACME++ versus ACME. The number of domain identifiers (N_i) in each *newOrder* request varies from 1, 5, 10, and 50, up to the maximum of 100 allowed per certificate by Let's Encrypt. Each configuration is repeated 10 times, and the average overhead ratios are computed.

4.4.2 Evaluation Results. Figures 6a and 6b present the simulation results for time and traffic overhead in the ACME++ process. For time overhead, the *Valid Authz* scenario incurs an average additional overhead of approximately 50%, while the *Pending Authz* scenario experiences about 20%. The time overhead is more variable with smaller numbers of domain identifiers (e.g., $N_i = 1$ or $N_i = 5$), likely due to network variability. However, as N_i increases, the time overhead stabilizes, indicating more consistent performance. Although the maximum time overhead reaches approximately 60%, the actual completion time of ACME++ typically remains only a few seconds when N_i is small.

For traffic overhead, a clear decreasing trend emerges for both the *Valid Authz* and *Pending Authz* scenarios as N_i increases. This suggests that the relative impact of the traffic diminishes with larger N_i, as the fixed communication costs are distributed over more identifiers, resulting in a lower per-identifier traffic overhead ratio. When $N_i = 100$, the traffic overhead is less than 10%. Although the ratio reaches approximately 50% when N_i is small, the total number of bytes transmitted is typically small—often only a few kilobytes—which has a negligible impact on network bandwidth.

In summary, even under conditions simulating maximum overhead, ACME++ maintains a time overhead below 60%, and the traffic overhead consistently remains below 50%. These results demonstrate that ACME++ provides a scalable and efficient solution, even when managing a large number of domain identifiers.

4.5 Attack Resilience

In this section, we demonstrate that the ACME++ protocol provides strong security against various attack scenarios.

1. ACME Authz Caching Attack. In this scenario, following the assumptions outlined in Section 3.1, the attacker has full knowledge of the victim's ACME account credentials, including the ACME++ client's IP address and ID, and attempts to exploit cached *Client Authz* and *Authz* records to obtain fraudulent certificates.

Feasibility Analysis: If the attacker uses their own IP address to communicate with the CA, the CA will detect a mismatch between the submitted client's IP and the attacker's IP, causing the CA to terminate the issuance process. If the attacker attempts to spoof the IP address to appear as the victim's, no response from the CA will be received unless one of the following two scenarios occurs:

(1) The attacker can control the victim's network traffic through methods such as BGP prefix hijacking, which contradicts the assumption that the attacker controls only a single host.
(2) The attacker shares the same IP address as the victim. However, this contradicts our attack assumptions, which assume that the attacker is typically distant from the victim and has identified the victim through internet scans. In this case, controlling another machine with the same IP as the victim is non-trivial. One possible exception could be cloud environments, where the attacker and victim may have virtual machines with the same IP. However, cloud providers allocate IP addresses randomly, making IP collisions highly unlikely.

2. Brute Force Attack. In this scenario, we assume the attacker shares the ACME account with the victim and has access to the victim's ACME account key and UID. The attacker attempts to obtain fraudulent certificates for domains controlled by other users within the same ACME account.

Feasibility Analysis: In this case, the attacker would need to guess the client ID. If the client ID consists of uppercase and lowercase letters, as well as digits, and has a length of 8 characters, the complexity of a brute-force attack would be $62^8 \approx 2.18 \times 10^{14}$ possible combinations, making such an attack infeasible.

3. Partial Domain Control Attack. In this scenario, we assume the attacker shares the ACME account with the victim and has control over some of the victim's domains. The attacker aims to obtain a fraudulent certificate that includes other victim domains.

Feasibility Analysis: During the client authorization process, which involves n challenges, the attacker can only succeed if all n challenges correspond to domains under their control. Since the domains for the challenges are selected randomly, the attacker would need significant control over the victim's domain portfolio to fulfill this condition. If the attacker has sufficient access to meet this requirement, they would likely already possess substantial control over the victim's infrastructure, rendering the attack scenario less practical and potentially redundant.

5 Related Work

5.1 Web PKI Threat Model

5.1.1 Private Key Compromise. The Web PKI ecosystem, which relies on cryptographic protocols and SSL certificates, is heavily dependent on secure private key management to maintain communication integrity. Early studies explored the risks associated with private key leakage due to server-side vulnerabilities, such as the 2008 Debian OpenSSL flaw [49], which resulted in the generation of weak or repeated keys, and the 2014 Heartbleed vulnerability [54], which exposed private keys through server memory leakage. Subsequent research by Cangialosi *et al.* [10] examined key-sharing scenarios involving certificate users and third-party hosting providers, including Content Delivery Networks (CDNs). More recently, Ma *et al.* [34] introduced the concept of *Certificate Invalidation Events*, where entities such as former domain owners or CDN providers continued to retain valid certificate keys despite changes in domain ownership.

While much of the prior work has focused on the risks associated with SSL certificate private keys, our research shifts attention to the vulnerabilities of compromised ACME account keys. We demonstrate that the compromise of an ACME account key has significant implications for the Web PKI ecosystem, as it enables attackers to issue entirely new, valid fraudulent certificates. In contrast, the compromise of certificate keys is often mitigated through the prompt revocation of the affected certificates.

5.1.2 Fraudulent Certificate Retrieval. To perform a MITM attack, attackers must first acquire a fraudulent certificate that matches the victim's identity. Previous research has developed several methods for obtaining fraudulent certificates within the Web PKI framework, primarily targeting DV processes [6, 8, 9, 17]. Birge-Lee *et al.* [6] demonstrated an attack in which BGP prefix hijacking successfully deceived DV processes and allowed the attacker to obtain a fraudulent certificate. Other notable attacks focus on DNS-based vulnerabilities. Borgolte *et al.* [8] exploited dangling DNS records in cloud environments to acquire fraudulent certificates by exploiting unallocated IP addresses. Similarly, Brandt *et al.* [9] introduced a DNS cache poisoning attack, where fragmented responses to a DNS resolver were manipulated to redirect the victim domain to an attacker-controlled IP address. Dai *et al.* [17] proposed off-path attacks on nameservers to manipulate the DV process, forcing the CA to rely on attacker-specified nameservers.

In contrast to these approaches, which involve deceiving the CA to the attacker-controlled IP addresses, our attack bypasses the DV process entirely by exploiting the ACME *Authz* caching mechanism. Once an attacker gains access to the victim's ACME account credentials, they can directly communicate with the CA to request a fraudulent certificate without triggering any DV. This method significantly reduces the attacker's overhead, eliminating the need for DNS or network-level manipulation, and lowers the barrier for attackers.

5.2 CT Information Leakage

In our attack model, CT logs provide a wealth of information, enabling attackers to identify *associated domains*. As CT logs continue to expand, concerns regarding potential information leakage and privacy violations have increasingly been raised [38, 41, 42, 44]. Scheitle *et al.* [42] conducted a heuristic analysis of both the positive and negative aspects of CT, emphasizing how these logs expose Fully Qualified Domain Names (FQDNs) and subdomains that would otherwise remain hidden from public scans. Kales *et al.* [29] expanded upon this by identifying sensitive information, such as usernames, email addresses, and business relationships, that could be inferred from CT logs. More recently, Pletinckx *et al.* [38] demonstrated how attackers could exploit CT logs to identify domains that have ceased renewing their certificates, making them more susceptible to known security threats.

In our work, we leverage publicly available CT logs to identify *associated domains* linked to the victim's ACME account, thereby expanding the attack surface. This approach highlights the dual nature of CT logs: while they are intended to enhance transparency and security, they can also inadvertently assist attackers in reconnaissance efforts.

6 Conclusion

In this work, we highlight a critical vulnerability in the ACME protocol that enables attackers to obtain fraudulent certificates by exploiting the *Authz* cache mechanism. We introduce the ACME *Authz* Cached Attack and demonstrate that Let's Encrypt is susceptible to this form of attack. In response, we propose ACME++, an enhanced protocol that incorporates *Client Authz* to strengthen client authorization. ACME++ mitigates the risk of unauthorized certificate issuance without requiring modifications to the existing Web PKI infrastructure or introducing significant overhead for CAs. We hope that this work raises awareness of this security threat within the CA, ACME client, and Web PKI communities, thereby encouraging the adoption of appropriate mitigations and further enhancing the robustness and trustworthiness of Web PKI.

Acknowledgments

We thank all anonymous reviewers for their valuable comments and suggestions. The research is supported by the National Natural Science Foundation of China under Grant No. 62394322.

References

[1] acmesh-official. 2024. acme.sh. https://github.com/acmesh-official/acme.sh Accessed: 2024-09-25.

[2] John Althouse. 2023. Easily Identify Malicious Servers on the Internet with JARM. https://engineering.salesforce.com/easily-identify-malicious-servers-on-the-internet-with-jarm-e095edac525a/ Accessed: 2024-09-28.

[3] Apache Software Foundation. 2025. Apache Log4j 2. https://logging.apache.org/log4j/2.x/index.html. Accessed: 2025-01-26.

[4] Trust Asia. 2020. Extremely Dangerous! Tesla Suffers Major Failure Due to Expired Certificates, Causing Large-Scale Outage. https://www.trustasia.com/view-tesla-expired/

[5] R. Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten. 2019. Automatic Certificate Management Environment (ACME). RFC 8555. https://doi.org/10.17487/RFC8555

[6] H. Birge-Lee, Y. Sun, A. Edmundson, J. Rexford, and P. Mittal. 2018. Bamboozling Certificate Authorities with BGP. In *Proceedings of the 27th USENIX Conference on Security Symposium* (Baltimore, MD, USA) *(SEC'18)*. USENIX Association, USA, 833–849.

[7] Google Security Blog. 2015. Maintaining Digital Certificate Security. https://security.googleblog.com/2015/03/maintaining-digital-certificate-security.html Accessed: 2024-10-14.

[8] K. Borgolte, T. Fiebig, S. Hao, C. Kruegel, and G. Vigna. 2018. Cloud Strife: Mitigating the Security Risks of Domain-Validated Certificates. In *Proceedings of the 2018 Applied Networking Research Workshop* (Montreal, QC, Canada) *(ANRW '18)*. Association for Computing Machinery, New York, NY, USA, 4. https://doi.org/10.1145/3232755.3232859

[9] M. Brandt, T. Dai, A. Klein, H. Shulman, and M. Waidner. 2018. Domain Validation++ For MitM-Resilient PKI. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 2060–2076. https://doi.org/10.1145/3243734.3243790

[10] F. Cangialosi, T. Chung, D. Choffnes, D. Levin, B. M. Maggs, A. Mislove, and C. Wilson. 2016. Measurement and Analysis of Private Key Sharing in the HTTPS Ecosystem. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) *(CCS '16)*. Association for Computing Machinery, New York, NY, USA, 628–640. https://doi.org/10.1145/2976749.2978301

[11] Checkmarx. 2024. Zed Attack Proxy (ZAP). https://github.com/zaproxy/zaproxy. Accessed: 2025-01-28.

[12] Weihan Chen, Zhiliang Wang, Han Zhang, Xia Yin, and Xingang Shi. 2021. Cost-Efficient Dynamic Service Function Chain Embedding in Edge Clouds. In *2021 17th International Conference on Network and Service Management (CNSM)*. 310–318. https://doi.org/10.23919/CNSM52442.2021.9615590

[13] L. Chuat, A. Abdou, R. Sasse, C. Sprenger, D. Basin, and A. Perrig. 2020. SoK: Delegation and Revocation, the Missing Links in the Web's Chain of Trust. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Genova, Italy, 624–638. https://doi.org/10.1109/EuroSP48549.2020.00046

[14] J. Clark and P. C. van Oorschot. 2013. SoK: SSL and HTTPS: Revisiting Past Challenges and Evaluating Certificate Trust Model Enhancements. In *2013 IEEE Symposium on Security and Privacy*. IEEE, Berkeley, CA, USA, 511–525. https://doi.org/10.1109/SP.2013.41

[15] Cloudflare. 2024. Cloudflare Nimbus Certificate Transparency Logs. https://ct.cloudflare.com/logs/nimbus2024/ Additional logs available at https://ct.cloudflare.com/logs/nimbus2025/ Accessed: 2024-10-14.

[16] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk. 2008. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. Technical Report RFC 5280. Network Working Group. https://www.rfc-editor.org/rfc/rfc5280

[17] T. Dai, H. Shulman, and M. Waidner. 2021. Let's Downgrade Let's Encrypt. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 1421–1440. https://doi.org/10.1145/3460120.3484815

[18] DigiCert. 2024. Third-Party ACME Integration Guide. https://docs.digicert.com/en/certcentral/certificate-tools/certificate-lifecycle-automation-guides/third-party-acme-integration.html Accessed: 2024-10-14.

[19] Electronic Frontier Foundation. 2024. Certbot. https://certbot.eff.org/ Accessed: 2024-09-25.

[20] Let's Encrypt. 2024. Rate Limits. https://letsencrypt.org/docs/rate-limits/ Accessed: 2024-10-06.

[21] Toptal Talent Network Experts. 2024. 10 Common Web Security Vulnerabilities. https://www.toptal.com/cybersecurity/10-most-common-web-security-vulnerabilities. Accessed: 2025-01-28.

[22] S. M. Farhan and T. Chung. 2023. Exploring the Evolution of TLS Certificates. In *Passive and Active Measurement: 24th International Conference, PAM 2023, Virtual Event, March 21–23, 2023, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 71–84. https://doi.org/10.1007/978-3-031-28486-1_4

[23] Dennis Fisher. 2012. Final Report on DigiNotar Hack Shows Total Compromise of CA Servers. https://threatpost.com/final-report-diginotar-hack-shows-total-compromise-ca-servers-103112/77170/. Accessed: August 18, 2024.

[24] Haijun Geng, Xingang Shi, Xia Yin, Zhiliang Wang, and Han Zhang. 2015. Algebra and algorithms for efficient and correct multipath QoS routing in link state networks. In *2015 IEEE 23rd International Symposium on Quality of Service (IWQoS)*. 261–266. https://doi.org/10.1109/IWQoS.2015.7404744

[25] Google. 2024. Certificate Transparency. https://certificate.transparency.dev/. Accessed: August 18, 2024.

[26] Google. 2024. Google CT Argon 2024 Log. https://ct.googleapis.com/logs/us1/argon2024/ Accessed: 2024-10-14.

[27] Google. 2024. HTTPS Encryption on the Web. https://transparencyreport.google.com/https/overview. Accessed: August 18, 2024.

[28] David Hamann. 2022. nginx alias misconfiguration allowing path traversal. https://davidhamann.de/2022/08/14/nginx-alias-traversal/ Updated: August 14, 2022.

[29] D. Kales, O. Omolola, and S. Ramacher. 2019. Revisiting User Privacy for Certificate Transparency. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. 432–447. https://doi.org/10.1109/EuroSP.2019.00039

[30] Salabat Khan, Fei Luo, Zijian Zhang, Farhan Ullah, Farhan Amin, Syed Furqan Qadri, Md Belal Bin Heyat, Rukhsana Ruby, Lu Wang, Shamsher Ullah, Meng Li, Victor C. M. Leung, and Kaishun Wu. 2023. A Survey on X.509 Public-Key Infrastructure, Certificate Revocation, and Their Modern Implementation on Blockchain and Ledger Technologies. *IEEE Communications Surveys & Tutorials* 25, 4 (2023), 2529–2568. https://doi.org/10.1109/COMST.2023.3323640

[31] B. Laurie, A. Langley, and E. Kasper. 2013. *Certificate Transparency*. Technical Report RFC 6962. Internet Engineering Task Force (IETF). https://www.rfc-editor.org/info/rfc6962

[32] Let's Encrypt. n.d.. Challenge Types. https://letsencrypt.org/docs/challenge-types/ Accessed: 2024-09-25.

[33] Let's Encrypt. n.d.. Let's Encrypt. https://letsencrypt.org/ Accessed: 2024-09-25.

[34] Z. Ma, A. Faulkenberry, T. Papastergiou, Z. Durumeric, M. D. Bailey, A. D. Keromytis, F. Monrose, and M. Antonakakis. 2023. Stale TLS Certificates: Investigating Precarious Third-Party Access to Valid TLS Keys. In *Proceedings of the 2023 ACM on Internet Measurement Conference* (Montreal QC, Canada) *(IMC '23)*. Association for Computing Machinery, New York, NY, USA, 222–235. https://doi.org/10.1145/3618257.3624802

[35] J. Nightingale. 2011. Fraudulent *.google.com Certificate. https://blog.mozilla.org/security/2011/08/29/fraudulent-google-com-certificate/. Accessed: August 18, 2024.

[36] OWASP. 2024. OWASP Top Ten. https://owasp.org/www-project-top-ten/. Accessed: 2025-01-28.

[37] OWASP. n.d.. Path Traversal Attack. https://owasp.org/www-community/attacks/Path_Traversal Accessed: 2024-09-25.

[38] S. Pletinckx, T.-D. Nguyen, T. Fiebig, C. Kruegel, and G. Vigna. 2023. Certifiably Vulnerable: Using Certificate Transparency Logs for Target Reconnaissance. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. 817–831. https://doi.org/10.1109/EuroSP57164.2023.00053

[39] PortSwigger. 2024. Burp Suite. https://www.kali.org/tools/burpsuite/. Accessed: 2025-01-28.

[40] R. Prodanović, I. Vulić, and I. Tot. 2019. A Survey of PKI Architecture. *5th International Conference – ERAZ 2019 – Knowledge Based Sustainable Development, Selected Papers* (2019), 169–176. https://doi.org/10.31410/ERAZ.S.P.2019.169 Conference held in Budapest, Hungary, May 23, 2019.

[41] R. Roberts and D. Levin. 2019. When Certificate Transparency Is Too Transparent: Analyzing Information Leakage in HTTPS Domain Names. In *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society* (London, United Kingdom) *(WPES'19)*. Association for Computing Machinery, New York, NY, USA, 87–92. https://doi.org/10.1145/3338498.3358655

[42] Q. Scheitle, O. Gasser, T. Nolte, J. Amann, L. Brent, G. Carle, R. Holz, T. C. Schmidt, and M. Wählisch. 2018. The Rise of Certificate Transparency and Its Implications on the Internet Ecosystem. In *Proceedings of the Internet Measurement Conference 2018* (Boston, MA, USA) *(IMC '18')*. Association for Computing Machinery, New York, NY, USA, 343–349. https://doi.org/10.1145/3278532.3278562

[43] Sectigo. 2024. Sectigo Sabre Certificate Transparency Logs. https://sabre2025h2.ct.sectigo.com/ Additional logs available at https://sabre2025h1.ct.sectigo.com/, https://sabre2024h2.ct.sectigo.com/, https://sabre2024h1.ct.sectigo.com/ Accessed: 2024-10-14.

[44] Junxian Shen, Han Zhang, Yantao Geng, Jiawei Li, Jilong Wang, and Mingwei Xu. 2022. Gringotts: Fast and Accurate Internal Denial-of-Wallet Detection for Serverless Computing *(CCS '22)*. Association for Computing Machinery, New York, NY, USA, 2627–2641. https://doi.org/10.1145/3548606.3560629

[45] Security Staff. 2024. 70% of Web Applications Have Severe Security Gaps. https://www.securitymagazine.com/articles/99770-70-of-web-applications-have-severe-security-gaps. Accessed: 2025-01-28.

[46] Dan Swinhoe. 2023. SpaceX Starlink Outage Caused by Expired Ground Station Certificates. https://www.datacenterdynamics.com/en/news/spacex-starlink-outage-caused-by-expired-ground-station-certificates/

[47] Dirty COW Team. 2016. Dirty COW (CVE-2016-5195). https://dirtycow.ninja/. Accessed: 2025-01-28.
[48] Cisco Umbrella. 2024. Umbrella Popularity List. https://umbrella-static.s3-us-west-1.amazonaws.com/index.html Accessed: 2024-10-06.
[49] S. Yilek, E. Rescorla, H. Shacham, B. Enright, and S. Savage. 2009. When Private Keys Are Public: Results from the 2008 Debian OpenSSL Vulnerability. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement (IMC)*. Association for Computing Machinery, New York, NY, USA, 15–27. https://doi.org/10.1145/1644893.1644896
[50] Chuqing Zhang, Han Zhang, and Xiaoting Hu. 2019. A Contrastive Study of Machine Learning on Funding Evaluation Prediction. *IEEE Access* 7 (2019), 106307–106315. https://doi.org/10.1109/ACCESS.2019.2927517
[51] Han Zhang, Haijun Geng, Yahui Li, Xia Yin, Xingang Shi, Zhiliang Wang, Qianhong Wu, and Jianwei Liu. 2019. DA&FD–Deadline-Aware and Flow Duration-Based Rate Control for Mixed Flows in DCNs. *IEEE/ACM Transactions on Networking* 27, 6 (2019), 2458–2471. https://doi.org/10.1109/TNET.2019.2951925
[52] Han Zhang, Xingang Shi, Yingya Guo, Zhiliang Wang, and Xia Yin. 2017. More load, more differentiation — Let more flows finish before deadline in data center networks. *Computer Networks* 127 (2017), 352–367. https://doi.org/10.1016/j.comnet.2017.08.020
[53] Han Zhang, Xingang Shi, Xia Yin, Jilong Wang, Zhiliang Wang, Yingya Guo, and Tian Lan. 2021. Boosting bandwidth availability over inter-DC WAN. In *Proceedings of the 17th International Conference on Emerging Networking EXperiments and Technologies* (Virtual Event, Germany) *(CoNEXT '21)*. Association for Computing Machinery, New York, NY, USA, 297–312. https://doi.org/10.1145/3485983.3494843
[54] L. Zhang, D. Choffnes, D. Levin, T. Dumitraș, A. Mislove, A. Schulman, and C. Wilson. 2014. Analysis of SSL Certificate Reissues and Revocations in the Wake of Heartbleed. In *Proceedings of the 2014 Conference on Internet Measurement Conference* (Vancouver, BC, Canada) *(IMC '14)*. Association for Computing Machinery, New York, NY, USA, 489–502. https://doi.org/10.1145/2663716.2663758

A Assumptions Further Explained with Examples

In this section, we provide additional examples and details to further clarify our assumptions regarding the acquisition of a victim's ACME account credentials.

Example 1: Path Traversal Vulnerability

If the web server process operates under the same user account as the private key file, or is initiated with root privileges, a path traversal vulnerability [37] could allow an attacker to access the private key, even when it is protected by restrictive permission (e.g., 600). Such vulnerabilities can arise from server misconfigurations [28]. This scenario could enable attackers to gain unauthorized access to ACME account credentials.

Example 2: Remote Code Execution (RCE) and Privilege Escalation

RCE vulnerabilities in web servers, such as the Apache Log4j vulnerability [3], can be exploited by attackers to execute arbitrary code. By leveraging privilege escalation exploits, such as the Dirty COW [47], attackers can elevate their privileges and gain access to sensitive account credentials. However, we assume that the attacker will not directly request certificates from the victim's machine. If the attacker was to do so, the victim could detect the anomaly and revoke the fraudulent certificate. Instead, it is more likely that the attacker would initiate the certificate request from their own machine, avoiding direct interaction with the victim's system.

Vulnerabilities Are Quite Common

Web server misconfigurations and vulnerabilities are prevalent. Reports from OWASP [36] and other security analyses [21, 45] continue to highlight common web security risks. Additionally, attackers can utilize web vulnerability scanning tools, such as OWASP ZAP [11] and Burp Suite [39], to conduct large-scale internet scans with minimal effort.

B Recovering Account UID Through Let's Encrypt's Weak ACME Implementation

In Section 3.2.1, we established that an attacker does not need to rebuild the account if the attacker has full access to the account credentials. However, in some scenarios, an attacker may exploit server-side vulnerabilities to obtain only the victim's ACME account private key but not the account UID. Such cases can arise if the attacker acquires the key pair through insecure key sharing or reuse [10], or due to vulnerabilities like Heartbleed [54].

In these cases, the attacker must identify the correct UID to reconstruct the victim's account profile. The attacker can exploit a weakness in Let's Encrypt's ACME implementation. According to the ACME RFC, account UIDs should be random strings; however, in Let's Encrypt's implementation, account UIDs are assigned incrementally as integers. This allows attackers to infer the valid UID by brute-forcing the potential range of UIDs. To do this, the attacker sends a POST-as-GET request to `/acme/acct/<guess_id>` with a guessed integer UID to the CA server. If the UID matches the victim's ACME account key, the server responds with an HTTP 200 OK and returns the account object. Otherwise, an error is returned.

The attacker can further narrow down the potential range of account UIDs by examining the website's certificate, specifically the `not_before` field, which indicates the certificate's issuance time. Since ACME typically completes account registration, certificate requests, and issuance within seconds, the attacker can estimate the registration time with second-level accuracy. This provides a strong basis for estimating the account UID range, significantly reducing the number of brute-force attempts required.